

CS2104: Programming Language Concepts
Audit
Summary

Sim Ray En Ryan

December 14, 2025

Contents

1	Overview (Lecture 1)	4
1.1	Course Objectives	4
1.2	Programming Paradigms	4
1.3	Advanced Languages Featured	4
1.3.1	Haskell (Purely Functional Language)	4
1.3.2	OCaml (Versatile Language)	4
1.3.3	Scala	5
1.3.4	Prolog (Logic)	5
2	Functions, Binders and Layout Rule (Lecture 2)	5
2.1	Declarative Programming Concepts	5
2.2	Writing Functions (Haskell Examples)	5
2.3	Curried vs Tupled Functions	6
2.4	Local Bindings (Binders)	6
2.5	Layout Rules (Haskell)	6
3	Data Types and Control Constructs (Lecture 3)	6
3.1	Primitive Types	6
3.2	User-Defined Types	7
3.2.1	Ordinal Types (Enumeration)	7
3.2.2	Union Types / Algebraic Data Types (ADT)	7
3.2.3	Record Types	7
3.2.4	Pointer Types	7
3.3	Type Classes (Haskell)	7
3.4	Efficient Recursion	8
4	Higher-Order Functions (Lecture 4)	8
4.1	First-Class Functions and Lazy Evaluation	8
4.2	Genericity (Fold and Map)	8
4.3	Composition and Piping	9
5	Other Haskell Features (Lecture 5)	9
5.1	List Comprehension (Syntactic Sugar)	9
5.2	Type Classes for Numbers and Arrays	9
5.2.1	Numbers Hierarchy	9
5.2.2	Arrays	9
5.3	Monoids	9
6	Towards Monads (Lecture 6)	10
6.1	Referential Transparency and Context	10
6.2	Higher-Order Type Classes	10
6.2.1	Functor	10

6.2.2	Applicative	10
6.2.3	Monad	10
7	Monadic Parsing with Haskell (Lecture 7)	10
7.1	Language Structure	10
7.2	Grammar Formalisms	11
7.3	Parser Combinators	11
8	Prolog and Constraint Logic Programming (Lecture 8-9)	11
8.1	Core Concepts	11
8.2	Unification and Resolution	11
8.3	List Manipulation and Arithmetic	11
8.4	Negation and Constraint Logic Programming (CLP)	12
9	OCaml Basics and Imperative Programming (Lecture 10)	12
9.1	OCaml vs Haskell (Comparison)	12
9.2	Language Basics	12
9.3	Mutability	12
9.4	Imperative Constructs	13
10	OOP and Modules in OCaml (Lecture 11)	13
10.1	Classes and Objects	13
10.2	Typing	13
10.3	Modules and Functors	13
11	Scala Highlights (Lecture 12-13)	14
11.1	Core Principles	14
11.2	Types and Classes	14
11.2.1	Classes and Abstract Classes	14
11.2.2	Unified and Polymorphic Types	14
11.3	Functional Constructs	14
11.4	Pattern Matching and Traits	14
11.5	Implicits and Inference	15

1 Overview (Lecture 1)

1.1 Course Objectives

- Cover key concepts in programming languages.
- Knowledge of language paradigms.
- Support for software abstraction, reuse and safety.
- Improving your programming skills.
- Make learning a new programming language easier.
- Highlight successful/advanced languages.

1.2 Programming Paradigms

- Imperative Programming.
- Functional Programming.
- Logic Programming.
- Object-Oriented Programming.
- Constraint Programming.
- Event-Driven Programming (not covered).
- Aspect-Oriented Programming (not covered).

1.3 Advanced Languages Featured

1.3.1 Haskell (Purely Functional Language)

- A Purely Functional Language featuring static typing, higher-order functions, polymorphism, type classes and monadic effects.
- Pure and Lazy Language.
- Advantages: concise, abstract, pure.
- Example: `inc :: Int -> Int` defined as `inc x = x + 1`.

1.3.2 OCaml (Versatile Language)

- Typeful higher-order functional and object-oriented language.
- Supports for stateful imperative programming.
- Powerful module system.

- Advantages: versatile, abstract, easy reuse.
- Example: `let inc x = x + 1.`

1.3.3 Scala

- Stands for "scalable language".
- A multi-paradigm language.
- Interoperates with all Java libraries.
- Advantages: concise, Java-compliant.

1.3.4 Prolog (Logic)

- Based on first-order logic.
- Used to define "relations" and relies on unification for execution.
- Advantages: relations, logic and unification.
- Example: `inc(X, Res):- Res is X+1..`

2 Functions, Binders and Layout Rule (Lecture 2)

2.1 Declarative Programming Concepts

- **Philosophy:** Say what you want to compute, and let the computer find how to compute it.
- **Properties:** Focus on functions (or relations). Stateless programming (no update of data structures).
- **Pure Functions:** Mathematical functions whose outputs depend solely on their inputs. Haskell is one of the purest languages.

2.2 Writing Functions (Haskell Examples)

- **Factorial (Bounded Integer):**

```
fact :: Int -> Int
fact n = if n==0 then 1
         else n * (fact (n-1))
```

- **Factorial (Infinite Precision):** Integer denotes integers of arbitrary precision.

```
fact2 :: Integer -> Integer
fact2 0 = 1
fact2 n = n * fact2 (n-1)
```

- **Type Conversion:** Use `fromIntegral` to fix type error in mixed precision calculations.

2.3 Curried vs Tupled Functions

- **Curried function:** Takes one argument at a time, by returning a function that takes the next argument.
- **Tupled function:** Takes all its arguments as a tuple/product.
- Curried functions support partial application naturally.

2.4 Local Bindings (Binders)

- **Definition:** Constructs that declare new identifiers and their possible values.
- **Haskell:** `let` binding is **recursive** by default. Local binders are **immutable**.
- **OCaml:** `let` binder is **non-recursive** by default. Requires `let rec` for recursive definitions.
- **C:** Local binder is **mutable** and non-recursive.

2.5 Layout Rules (Haskell)

- Uses indentation to structure code and minimize use of separators.
- Declarations are "aligned".
- Rule: The next character after keywords `where/let/of/do` determines the starting column for declarations.
- Declarations are mutual recursive by default.

3 Data Types and Control Constructs (Lecture 3)

3.1 Primitive Types

- Directly implemented in hardware (usually).
- **Haskell:** Types are quite high level and are essentially boxed types (like objects in Java).

- **Unboxed types** (`Int\#`) are rarely used except by compiler writers to optimize.

3.2 User-Defined Types

3.2.1 Ordinal Types (Enumeration)

- A finite enumeration of distinct values.
- **Haskell:** Implemented as a special case of algebraic data type, e.g., `data DaysObj = Mon | Tue | ...` deriving `Enum`, `Eq`, `Show`.
- **Scala:** Enumeration can be done through sealed case objects.

3.2.2 Union Types / Algebraic Data Types (ADT)

- **C's union:** Stores different type of values within the same memory space but at different times/instances.
- **Haskell's ADT:** Uses data constructor tags for type-safe union types.
- **Sum Type:** Similar to disjunction $\alpha \vee \beta$. e.g., type `Either a b = Left a | Right b`.
- **Product Type:** Similar to conjunction $\alpha \wedge \beta$. e.g., type `Pair a b = (a, b)`.

3.2.3 Record Types

- A collection of values of possibly different types with field names.
- **Haskell:** An extension to ADT; field names automatically become access methods.
- **OCaml:** A separate type; fields are immutable by default, but mutable fields are also permitted.

3.2.4 Pointer Types

- **C:** Captures the address of an object value. Crucial for recursive data types.
- **Haskell:** No explicit pointer types needed; recursive types are possible because each ADT is implicitly a pointer to a boxed value.

3.3 Type Classes (Haskell)

- Supports **ad-hoc polymorphism** (overloading) for a class of types.
- **Num Class:** Example to provide a common type for `+` operator across integer, float, etc..
- **Eq Class:** Defines equality operators (`==`), (`/=`) for types that support equality checking.

- **Class Extension:** Supports extension from some base type class, e.g., `class (Eq a) => Ord a where`
- **Derived Instances:** Standard type classes can be automatically derived (e.g., deriving (Eq, Ord)).

3.4 Efficient Recursion

- **Naïve List Reverse:** `reverse (x:xs) = (reverse xs) ++ [x]` is inefficient due to repeated linear-time append operations ($O(n^2)$).
- **Efficient (Tail-Recursive) Reverse:** Achieves linear time $O(n)$ by using an accumulator parameter.
- **Tail-Recursion:** The recursive call is the last operation in the recursive invocation. It is equivalent to a loop.
- **Tupled Fibonacci (Efficient):** Returns `(fib n, fib (n-1))` to eliminate redundant calls. Linear time, but not tail-recursive.
- **Tail-Recursive Fibonacci:** Implemented by adding two extra accumulator parameters.

4 Higher-Order Functions (Lecture 4)

4.1 First-Class Functions and Lazy Evaluation

- **First-Class Functions:** Functions can be passed as arguments, returned as results, constructed at run-time, and stored in data structures.
- **Lazy Evaluation:** The default evaluation for Haskell. Evaluation is delayed to application.
- **Closure:** An expression e abstracted into a function $\lambda(\lambda) e$.
- **Strict Evaluation:** Marking a constructor field with `!` forces strict evaluation. It is also enforced using `el 'seq' e2`.

4.2 Genericity (Fold and Map)

- **Fold Right (foldr):** Reduces from the right.
- **Fold Left (foldl):** Reduces from the left. It is typically tail-recursive.
- **Foldable Class:** A type class that defines generic folding over various data structures.
- **Functor Class:** Defines generic mapping over various type constructors, providing `fmap :: (a -> b) -> f a -> f b`.

4.3 Composition and Piping

- **Composition (.)**: `let (.) g f x = g (f x)`.
- **Pipe Operator (|>) / Apply (\$)**: Infix operators used for function application. `\$` has a very weak precedence.

5 Other Haskell Features (Lecture 5)

5.1 List Comprehension (Syntactic Sugar)

- A useful shorthand for building list data structures.
- **Generators**: `[f x | x <- xs]`.
- **Guards**: `quicksort [y | y <- xs, y < x] ...`.

5.2 Type Classes for Numbers and Arrays

5.2.1 Numbers Hierarchy

- **Num Class**: Defines `(+)`, `(-)`, `(*)`.
- **Integral Class**: Supports division via `div/mod` operations.
- **Fractional Class**: Supports `/` operation.

5.2.2 Arrays

- **Ix Class**: Defines properties for index types (must be `Ord`).
- **Creation**: Arrays are built from bounds and an association list of indices and values.
- **Updates**: `//` performs incremental update, returning a modified array.
- **Accumulation**: `accumArray` supports combining multiple values for the same index.

5.3 Monoids

- **SemiGroup**: A type class with an associative binary operation `op`.
- **Monoid**: Extends `SemiGroup` and includes a unit element `unit`.

6 Towards Monads (Lecture 6)

6.1 Referential Transparency and Context

- **Referential Transparency:** An expression can be replaced by its value without changing program behavior.
- **Value in Context:** Values often reside in a context, e.g., `Maybe a` (error/success) or `IO a` (side-effects).

6.2 Higher-Order Type Classes

6.2.1 Functor

- Allows applying a normal function to a value wrapped in a context.
- `class Functor f where fmap :: (a -> b) -> f a -> f b.`

6.2.2 Applicative

- Used when the function itself is wrapped in a context.
- `class Functor f => Applicative f where (<*>) :: f (a -> b) -> f a -> f b.`

6.2.3 Monad

- Handles feeding a wrapped value (ma) into a function that returns a wrapped value ($a \rightarrow mb$).
- `class Monad m where (>>=) :: m a -> (a -> m b) -> m b; return :: a -> m a.`
- **IO Monad:** Models input/output actions as sequences.
- **Do Notation:** Syntactic sugar for sequencing monadic actions.

7 Monadic Parsing with Haskell (Lecture 7)

7.1 Language Structure

- **Language = Syntax + Semantics.**
- **Lexical Analyzer (Scanner):** Recognizes character sequences and produces tokens.
- **Parser:** Recognizes a token sequence and produces a statement representation (parse tree/AST).

7.2 Grammar Formalisms

- **Context-Free Grammars (CFG):** Defined by terminals T , non-terminals N , a start symbol σ , and rewriting rules Ω .
- **BNF/EBNF:** Common notations for CFG.
- **Regular Grammars:** A subset of CFG, suitable for Lexical Analysis.

7.3 Parser Combinators

- **Generic Parser Type:** type Parser $a = \text{Tokens} \rightarrow (a, \text{Tokens})$.
- **Monadic Combinators** are used to build bigger parsers.
- **Sequential** Combines parsers $p1 \sim p2$.
- **Alternative** Allows a choice between two parsers.
- **Repetitive:** (zero or more) and (one or more).

8 Prolog and Constraint Logic Programming (Lecture 8-9)

8.1 Core Concepts

- **Atoms, Variables, Terms:** Atoms are constants (start lower-case); Variables start upper-case or $_$. Terms form tree-like structures.
- **Relations vs Functions:** Prolog specifies relations, not functions.
- **Facts and Rules:** Relations defined by facts and Horn clauses (rules).
- **Recursive Clauses:** Can define recursive relations like ancestor.

8.2 Unification and Resolution

- **Unification (=):** Computes a substitution for variables to make two terms equal. It is a single-assignment property.
- **Resolution:** The process of answering a query. It involves renaming variables.

8.3 List Manipulation and Arithmetic

- **Lists:** Notation uses square brackets $[...]$. $[H|T]$ breaks the list into head and tail.
- **Arithmetic (is):** Evaluates an arithmetic expression. The 2^{nd} argument must be a ground expression (without variables) for evaluation.

8.4 Negation and Constraint Logic Programming (CLP)

- **Negation as Failure (not):** Assumes a closed world: whatever cannot be proven is false.
- **Cut (!):** An impure feature used to limit backtracking.
- **CLP (Finite Domain):** Constraint solving based on bounded arithmetic. Constraints use operators like `\#=`, `\#>`, `\#\backslash=`.

9 OCaml Basics and Imperative Programming (Lecture 10)

9.1 OCaml vs Haskell (Comparison)

OCaml	Haskell
Strict (by default)	Lazy (by default)
Impure	Pure
Impure I/O	Monadic I/O
Modules/Functors	Type classes
OOP	Layout Rule, Comprehension

9.2 Language Basics

- **Let Binding:** `let` is non-recursive; `let rec` is required for recursion.
- **Evaluation:** Arguments and `let` bindings are strictly evaluated.
- **Laziness:** Captured explicitly via type `'a Lazy.t`. Deferred computation is built using `lazy (e)`.
- **Sequences:** `e1; e2; ...` executes for effects.

9.3 Mutability

- **Mutable Reference (`'a ref`):** Created by `ref v`. Dereferenced by `!ptr`. Updated by `ptr := v`.
- **Mutable Fields:** Fields can be marked mutable in a record type definition. Only mutable fields can be updated.
- **Weak Polymorphism:** Mutable fields/references must not be polymorphic, requiring instantiation to a monotype.

9.4 Imperative Constructs

- **Loops:** `for i=... do ... done` and `while ... do ... done` are provided.
- **Mutable Modules:** Modules like `Array` and `Hashtbl` contain mutable structures.
- **Memoization:** Technique to eliminate redundant calls by tabulation, often using hash tables.

10 OOP and Modules in OCaml (Lecture 11)

10.1 Classes and Objects

- **Class Structure:** Consists of fields (`val mutable`) and methods (`method`).
- **Self Reference:** `object (s)` names the current object `s`.
- **Inheritance:** Achieved via `inherit`.
- **Polymorphism:** Supported using type variables, e.g., `class ['a] buffer`.
- **Virtual Classes/Methods:** `class virtual` and `method virtual` define abstract classes/methods.

10.2 Typing

- **Class Signature (`class type`):** Based on the set of visible methods.
- **Structural Typing:** Two object types are the same if they are structurally equivalent on their visible methods.
- **Row Polymorphism:** Supports structural subtyping.

10.3 Modules and Functors

- **Module (`module`):** Used to structure large programs.
- **Module Signature (`module type`):** The interface for a module.
- **Abstract Data Type (ADT):** Achieved by declaring a type as `abstract (type 'a t)` in the module signature, hiding its implementation.
- **Functor (`functor`):** A function from a structure to a structure. Used for code reuse and generating specialized modules.

11 Scala Highlights (Lecture 12-13)

11.1 Core Principles

- **Motivation:** Interoperable with Java, concise, supports OOP+FP, and has advanced static typing and inference.
- **Object-Oriented:** Every value is an object. Classes and traits describe types/behavior. Uses mixin composition instead of multiple inheritance.
- **Functional:** Every function is an object and a first-class value. Supports anonymous functions and pattern-matching via Case Classes.

11.2 Types and Classes

11.2.1 Classes and Abstract Classes

- Classes are instantiated with `new`. Overriding methods requires the `override` keyword.
- An `object` keyword declares a class with a single object (singleton).
- **Abstract Classes** can have deferred/abstract types (type `T`) and deferred value definitions (`val element: T`).

11.2.2 Unified and Polymorphic Types

- **Unified Types:** All values are objects. `scala.Any` is the superclass of all classes.
- **Polymorphic Methods:** Methods can be parameterized by both values and types.

11.3 Functional Constructs

- **Anonymous Functions:** Shorthand syntax, e.g., `(x: Int) => x + 1`.
- **Placeholder:** `\_` denotes a placeholder for parameters.
- **Currying:** Methods with multiple parameter lists yield a function for the remaining lists when partially applied.
- **Folding:** Supports both fold left (`z /: xs`) and fold right (`xs : \z`).
- **List Constructors:** `Nil` and `::` are the two basic constructors.

11.4 Pattern Matching and Traits

- **Pattern Matching:** Uses the `match` keyword with a first-match policy on any data type.

- **Case Classes:** Allow constructor parameters to be exported via pattern-matching. They automatically derive equality.
- **Traits (Mixins):** Similar to Java interfaces, but can have fields and methods (including default implementations). Used for mixin class composition.

11.5 Implicits and Inference

- **Implicit Conversion:** Implicit definitions perform automatic type casting, e.g., enabling automatic conversion of Double to Int.
- **Implicit Parameters:** Can customize default parameters. They support systematic method overloading.
- **Type Inference:** Automatically infers types for variables, non-recursive methods, and polymorphic methods.